

Exercícios – Estruturas de Repetição – **While / DoWhile e For**

As estruturas de repetição são muito utilizadas quando precisamos **executar um bloco de instruções várias vezes enquanto uma dada condição tenha o resultado verdadeiro (true)**.

Existem três tipos de estruturas de repetição: **while**, **do...while** e **for**.

As três estruturas possuem o mesmo objetivo, repetir as instruções do bloco de comandos até que uma dada condição retorne o resultado falso (false). No entanto, cada instrução possui uma sintaxe diferente e, dependendo do tipo de cenário ou regra que se deseja aplicar, a utilização de uma determinada estrutura pode ser considerada mais ideal.

Relembrando: Um **bloco de comando** consiste em um agrupamento de instruções JAVA dentro de uma estrutura. Estes blocos são delimitados pelos caracteres: “{” que representa o início do bloco e “}” que representa o fim do bloco de comando. A existência de um bloco de comando pode ser dispensada quando o procedimento tiver apenas uma instrução de comando. A seguir vamos verificar quais as diferenças e sintaxe de cada uma das estruturas de repetição.

1 - ESTRUTURA DE REPETIÇÃO: **While**

Para utilizarmos o **while** devemos utilizar a seguinte sintaxe:

```
while(condição) instrução; // utilizado quando temos apenas uma instrução

OU

while(condição) // utilizado quando temos apenas uma instrução
    instrução;

OU

while(condição){
    // precisamos do bloco de comando quando temos várias instruções para executar
    instrução1;
    instrução2;
    instruçãoN;
}
```

As duas primeiras sintaxes são utilizadas quando temos **apenas uma instrução Java a ser executada após a validação da condição**, portanto não é necessário criar um bloco de comando para agrupar e executar as instruções. Na terceira sintaxe, temos mais de uma instrução Java a ser executada, portanto, é necessária a criação de um bloco de comando para agrupar as instruções e executá-las conforme a sequência definida.

Da mesma forma que no **if** (estrutura de seleção) **a condição entre os parênteses do while devem possuir um valor verdadeiro para que as instruções sejam executadas**. Desta forma, somente se o resultado da condição for verdadeiro é que as instruções serão executadas.

Para utilizar uma estrutura de repetição, porém, é importante que em algum momento o valor da condição testado se torne falso, caso contrário, a aplicação ficará em eterno “loop”, ou seja, ficará executando as instruções eternamente. Isto acontece porque as instruções somente serão executadas enquanto o valor da condição for verdadeiro, então se a condição permanecer eternamente com o valor verdadeiro a aplicação estará executando as instruções infinitamente. Esta característica é comum para todas as estruturas de repetição.

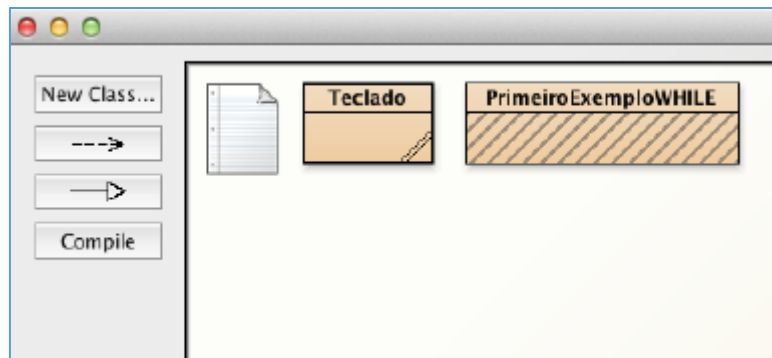
No caso específico do **while**, **a condição é testada no início da instrução while** e em seguida é feita a execução das instruções. Se a condição permanecer com o valor verdadeiro as instruções serão executadas novamente e assim sucessivamente até que o valor da condição se torne falso.

1.1 - PRIMEIRO EXEMPLO UTILIZANDO O **while**

Vamos iniciar a prática verificando como utilizar a estrutura de repetição: while em Java. Para tanto, abra o BlueJ e crie um novo projeto com o nome: **EstruturasDeRepeticao**.

Vamos utilizar a classe Teclado, para solicitar os dados do teclado.

Crie uma nova **classe de teste** chamada: **PrimeiroExemploWHILE**. Deixe a opção Class marcada e clique em OK. A classe criada será apresentada na tela do BlueJ conforme a figura abaixo.



Clique duas vezes em cima da classe para editá-la. Apague todo o conteúdo padrão deixando apenas a estrutura principal da classe de teste, conforme figura abaixo.

Vamos agora desenvolver a lógica do exemplo.

Para praticarmos os conceitos vamos utilizar como exemplo o cenário de um caixa eletrônico. Então, para testar o primeiro exemplo com o WHILE, vamos criar uma variável chamada: saldo e declarar um valor para esta variável. Depois, vamos criar uma outra variável chamada: retirada que irá ler do teclado um valor para retirada.

Caso o valor da retirada seja maior que o valor do saldo definido, vamos solicitar ao usuário que informe um valor inferior para retirada. Assim, ENQUANTO o usuário informar um valor superior ao valor definido no saldo, o sistema vai solicitar um novo valor de acordo com o saldo disponível.

Vamos desenvolver a lógica conforme a figura abaixo.

```
//classe de teste
public class PrimeiroExemploWHILE
{
    //método main da classe de teste
    //responsável pela execução dos comandos
    public static void main(String args[])
    {
        //criando a variavel saldo
        double saldo = 600.00;

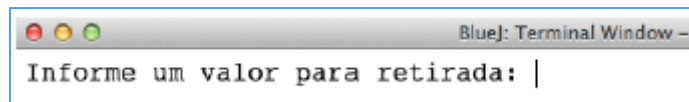
        //criando a variável retirada e lendo o valor do teclado
        double retirada = Teclado.leDouble("Informe um valor para retirada: ");

        //enquanto o valor da retirada for superior ao saldo
        //será solicitado para o usuário informar um novo valor para retirada
        while(retirada>saldo){
            System.out.println("Saldo indisponível para este valor de retirada.");
            retirada = Teclado.leDouble("Informe um novo valor para retirada: ");
        }

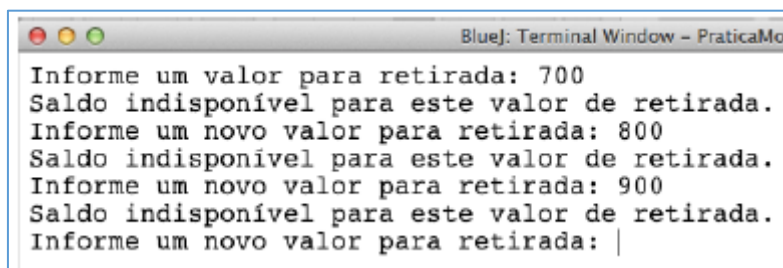
        //calcula novo saldo
        saldo = saldo - retirada;

        //apresenta novo saldo
        System.out.println("Seu novo saldo é: " + saldo);
    }
}
```

A partir de agora, podemos compilar a classe e executar o método main para ver o resultado da execução. Ao executar pela primeira vez, o sistema irá solicitar o valor para a retirada, conforme a figura abaixo:

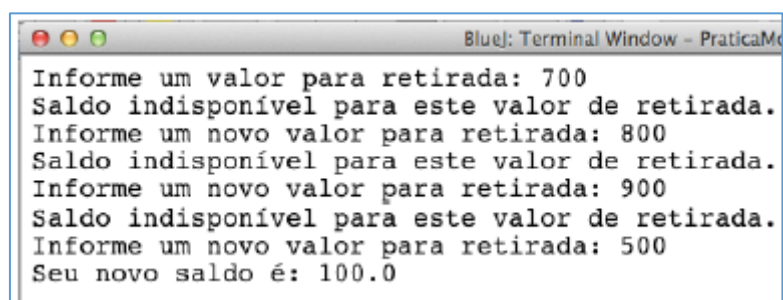


Se por acaso, **informarmos um valor superior ao valor do saldo definido**, o sistema irá apresentar a mensagem de saldo insuficiente e solicitar um novo valor. Veja na figura abaixo este comportamento:

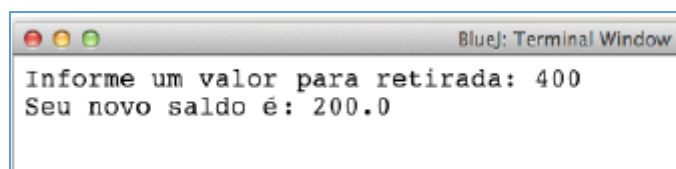


Analisando o resultado do teste, verificamos que **enquanto informarmos um valor superior ao saldo, o sistema ficará repetindo as instruções** que estão dentro do bloco de comandos do while.

Porém, quando informarmos um valor igual ou abaixo do valor definido para o saldo, o sistema finalizará a repetição e executará as demais instruções após o bloco de comando do while. A figura abaixo apresenta este comportamento.



Se executarmos novamente a classe de teste e modificarmos nosso cenário informando um valor igual ou abaixo do valor definido para o saldo, logo no início, as instruções que estão dentro do bloco de comandos do while nunca serão executadas. Isso ocorre porque **o while faz o teste da condição antes de executar qualquer instrução** e, neste caso, como o resultado da condição será falso, as instruções que estão abaixo do while jamais serão executadas. Veja abaixo a figura que ilustra este comportamento:



2- ESTRUTURA DE REPETIÇÃO: **do...while**

A estrutura de repetição **do...while** é semelhante a estrutura **while**. A única diferença entre elas é que **o while testa a condição no início** do comando e a estrutura **do...while testa a condição no final** do comando. Isto significa que se a condição estiver com o valor falso, utilizando **do...while** as instruções que estão dentro do bloco de comando desta estrutura serão executadas pelo menos uma vez. Já utilizando o **while**, se a condição estiver com o valor falso, as instruções não serão executadas.

Para utilizarmos o **do...while** em nossos programas devemos utilizar a seguinte sintaxe:

```
do // utilizado quando temos apenas uma instrução
    instrução;
while(condição);

OU

do{ // precisamos do bloco de comando quando temos várias instruções para executar
    instrução;
}while(condição);
```

2.2 SEGUNDO EXEMPLO UTILIZANDO O **do...while**

Vamos então criar uma nova classe para praticar o uso da estrutura de repetição: **do...while**.

Crie uma nova **classe de teste** chamada: **SegundoExemploDOWHILE** conforme figura abaixo.

Vamos utilizar o mesmo exemplo utilizado na prática do **while** para compararmos o comportamento das duas estruturas.

```
//classe de teste
public class SegundoExemploDOWHILE
{
    //método main da classe de teste
    //responsável pela execução dos comandos
    public static void main(String args[])
    {
        //criando a variavel saldo
        double saldo = 600.00;

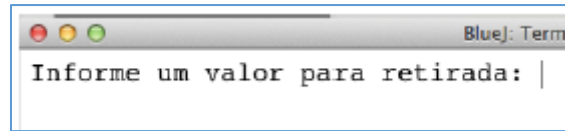
        //criando a variável retirada e lendo o valor do teclado
        double retirada = Teclado.leDouble("Informe um valor para retirada: ");

        //enquanto o valor da retirada for superior ao saldo
        //será solicitado para o usuário informar um novo valor para retirada
        do{
            System.out.println("Saldo indisponível para este valor de retirada.");
            retirada = Teclado.leDouble("Informe um novo valor para retirada: ");
        }while(retirada>saldo);

        //calcula novo saldo
        saldo = saldo - retirada;

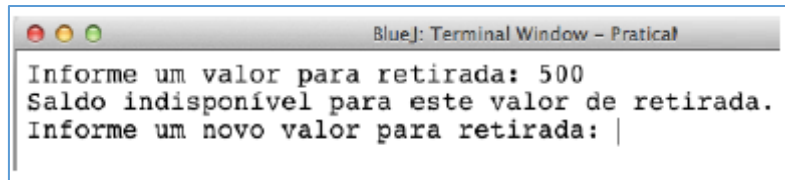
        //apresenta novo saldo
        System.out.println("Seu novo saldo é: " + saldo);
    }
}
```

Verifique que utilizamos exatamente o mesmo cenário para o teste. Modificamos apenas o uso da estrutura de repetição, ao invés do **while**, colocamos o **do...while**. Vamos então compilar e executar a classe de teste para verificar o seu comportamento.



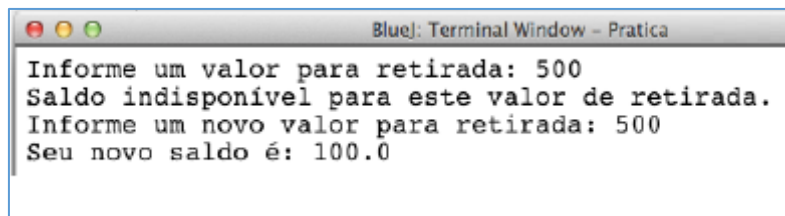
```
BlueJ: Termi
Informe um valor para retirada: |
```

Inicialmente o comportamento das duas classes de teste foi exatamente igual. Porém, se recordarmos o exemplo utilizado com o while, ao informarmos um valor igual ou inferior ao saldo as instruções que estão dentro do bloco de comandos do while, jamais serão executadas. Vamos então verificar como seria este comportamento utilizando o do...while. A figura abaixo ilustra este comportamento:



```
BlueJ: Terminal Window - Pratica
Informe um valor para retirada: 500
Saldo indisponível para este valor de retirada.
Informe um novo valor para retirada: |
```

Verifique que mesmo o valor sendo inferior ao valor do saldo, as instruções que estão dentro do do...while foram executadas. Vamos então informar novamente o valor 500 para verificar o que ocorre. A figura abaixo ilustra este comportamento:



```
BlueJ: Terminal Window - Pratica
Informe um valor para retirada: 500
Saldo indisponível para este valor de retirada.
Informe um novo valor para retirada: 500
Seu novo saldo é: 100.0
```

Analisando o resultado do teste acima, verificamos que as instruções que estão dentro do bloco de comandos do do...while foram apresentadas uma única vez. Por que isso ocorreu, visto que o valor informado para retirada era inferior ao valor do saldo? Isto ocorreu porque o **do...while faz o teste do resultado da condição somente no final** e, para este cenário, a sua utilização não é adequada.

Para este cenário o ideal é utilizar o while, visto que o teste do resultado da condição é realizado logo no início e, caso o valor esteja correto logo no início, as instruções dentro do bloco de comandos não precisam ser executadas.

2.3 TERCEIRO EXEMPLO UTILIZANDO O **do...while**

Vamos então utilizar um outro cenário, para o caixa eletrônico, onde o ideal é a utilização das estruturas: do...while. Crie uma nova **classe de teste** chamada: **TerceiroExemploDOWHILE** conforme figura abaixo.

```
//classe de teste
public class TerceiroExemploDOWHILE
{
    //método main da classe de teste
    //responsável pela execução dos comandos
    public static void main(String args[])
    {
        //criando a variavel saldo
        double saldo = 600.00;
        //criando uma variável opcao para verificar se o cliente deseja fazer uma nova operação
        char opcao;
        do{
            //criando a variável retirada e lendo o valor do teclado
            double retirada = Teclado.leDouble("Informe um valor para retirada: ");

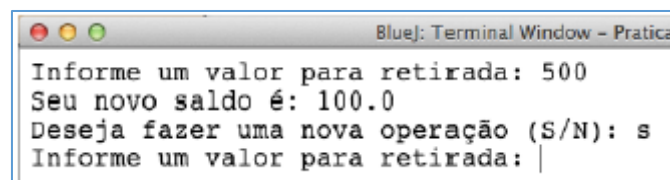
            //enquanto o valor da retirada for superior ao saldo
            //será solicitado para o usuário informar um novo valor para retirada
            while(retirada>saldo){
                System.out.println("Saldo indisponível para este valor de retirada.");
                retirada = Teclado.leDouble("Informe um novo valor para retirada: ");
            }

            //calcula novo saldo
            saldo = saldo - retirada;

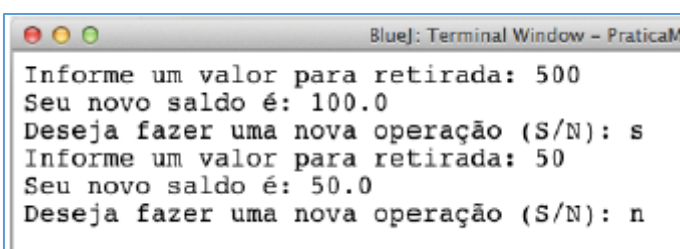
            //apresenta novo saldo
            System.out.println("Seu novo saldo é: " + saldo);

            //verifica se o usuário deseja fazer uma nova operação
            opcao = Teclado.leChar("Deseja fazer uma nova operação (S/N): ");
        }while(Character.toLowerCase(opcao)=='s');
    }
}
```

Neste novo exemplo vamos utilizar o do..while para verificar **se o usuário deseja continuar realizando a operação de retirada no caixa eletrônico**. Como no nosso cenário estamos trabalhando com a operação retirada, **no mínimo uma vez o usuário realizará esta operação**, portanto, para este caso, o uso do do...while é adequado pois as instruções que estão dentro do seu bloco de comandos serão executadas no mínimo uma vez. Depois disso, **as instruções somente serão executadas novamente**, em repetição, enquanto o usuário informar que deseja realizar uma nova operação, **variável opcao == 's'**, caso contrário, a repetição será encerrada. Vamos verificar este comportamento nas figuras abaixo:



```
BlueJ: Terminal Window - PraticaM
Informe um valor para retirada: 500
Seu novo saldo é: 100.0
Deseja fazer uma nova operação (S/N): s
Informe um valor para retirada: |
```



```
BlueJ: Terminal Window - PraticaM
Informe um valor para retirada: 500
Seu novo saldo é: 100.0
Deseja fazer uma nova operação (S/N): s
Informe um valor para retirada: 50
Seu novo saldo é: 50.0
Deseja fazer uma nova operação (S/N): n
```

Analisando o resultado do teste, na primeira figura, como informarmos o valor 's' para continuar a operação, o sistema repetiu novamente as instruções do bloco de comandos do do..while e permitiu uma nova retirada. Então, enquanto informarmos o valor = 's' este comportamento ocorrerá, porém, no momento que informarmos o valor = 'n' a repetição será encerrada, como apresentado na segunda figura.

Importante: neste terceiro exemplo utilizamos estruturas de repetição aninhadas. Fizemos uso do **do...while** e dentro dele do **while**. Podemos fazer uso de estruturas de repetição aninhadas dependendo do tipo de cenário que temos que desenvolver. **Este conceito de estrutura aninhada também pode ser utilizado com as estruturas de seleção.**

Diferença While x DoWhile

- Comando **while**: testa a condição, depois executa
- Comando **do-while**: executa, depois testa a condição



3 ESTRUTURA DE REPETIÇÃO: **for**

A estrutura de repetição **for** possui a mesma função que o **while** e o **do...while**, porém sua estrutura e sintaxe é bem diferente.

O **for** também testa uma condição e executa as instruções do bloco de comando caso o valor da condição seja verdadeiro, porém no **for** geralmente a condição é baseada em um contador. Por ter a sua condição geralmente baseada em uma variável contadora, o **for** é muito útil quando temos uma lista de valores para ser mostrado ou trabalhado e temos o conhecimento do início e fim desta lista. É imprescindível que saibamos o início e fim da lista, pois estes dados são utilizados na sintaxe do **for**.

Para utilizarmos o **for** em nossos programas devemos utilizar a seguinte sintaxe:

```
for(inicialização do contador;condição;incremento ou decremento do contador)
    instrução;

Ou utilizando bloco de comandos:

for(inicialização do contador;condição;incremento ou decremento do contador){
    instrução1;
    instruçãoN;
}
```

3.1 QUARTO EXEMPLO UTILIZANDO O **for**

Vamos então criar uma nova classe para praticar o uso do **for**. Crie uma nova **classe de teste** chamada: **QuartoExemploFOR** conforme figura abaixo.

```
//classe de teste
public class QuartoExemploFOR
{
    //método main da classe de teste
    //responsável pela execução dos comandos
    public static void main(String args[])
    {
        //criando a variável emprestimo e lendo o valor do teclado
        double emprestimo = Teclado.leDouble("Informe um valor para o emprestimo: ");

        //criando a variavel parcelas e lendo o valor do teclado
        double parcelas = Teclado.leInt("Informe a qtd de parcelas para o emprestimo (1-12): ");

        //criando a variavel valor da parcela
        double valorParcela = emprestimo/parcelas;

        //utilizando o for para calcular o valor das parcelas com 5% de acrescimo e apresentar para o usuário
        for(int contador = 1; contador <= parcelas; contador++){
            valorParcela += valorParcela*0.05;
            System.out.printf("O valor da parcela %d é: %4.2f \n", contador, valorParcela);
        }
    }
}
```

Neste exemplo, estamos realizando a operação de empréstimo onde o usuário informa o valor que deseja emprestar e a quantidade de parcelas que deseja utilizar para o pagamento. Para este cenário, utilizamos o **for** para realizar o cálculo de cada parcela e apresentar para o usuário, visto que cada parcela terá o acréscimo de 5%. Neste caso, como sabemos o início e o fim da repetição, a utilização do **for** é ideal, visto que podemos utilizar o contador do **for** para nos auxiliar na quantidade de repetições necessárias. Ao compilarmos e executarmos esta classe teremos o seguinte resultado:

```
BlueJ: Terminal Window - PraticaM
Informe um valor para o emprestimo: 500
Informe a qtde de parcelas para o emprestimo (1-12): 5
O valor da parcela 1 é: 105.00
O valor da parcela 2 é: 110.25
O valor da parcela 3 é: 115.76
O valor da parcela 4 é: 121.55
O valor da parcela 5 é: 127.63
```

4 – Praticando com “Classes”, “Classes de Teste” e “Estruturas de Repetição”

Neste exemplo, vamos criar a classe: **ContaCorrente**, para exemplificar o uso das estruturas de repetição dentro dos métodos de uma classe. A classe possuirá a seguinte estrutura:

1. Atributo saldo para controlar o saldo da conta;

```
public class ContaCorrente
{
    //atributos da classe
    private double saldo;
```

2. Método construtor para inicializar o saldo da conta;

```
public class ContaCorrente
{
    //atributos da classe
    private double saldo;

    //construtor da Classe
    public ContaCorrente(double s)
    {
        saldo = s;
    }
```

3. Método retirada para permitir o saque da conta. Neste método vamos controlar o valor do saque, para não permitir valores superiores ao valor do saldo;

```
//método retirada
public void retirada(double valor)
{
    //utilizando o while para validar o valor, deve ser inferior ao saldo
    while(valor>saldo){
        System.out.print("Saldo indisponível para retirada.");
        valor = Teclado.leDouble("Informe um novo valor para retirada: ");
    }
    saldo-=valor;
}
```

4. Método depósito para permitir depósitos na conta. Neste método também vamos controlar o valor do depósito, para não permitir valores igual ou abaixo de zero;

```
//método deposito
public void deposito(double valor)
{
    //utilizando o while para validar o valor, deve ser superior a zero (0)
    while(valor<=0){
        System.out.print("Valor incorreto para depósito.");
        valor = Teclado.leDouble("Informe um novo valor para depósito: ");
    }
    saldo+=valor;
}
```

5. Método empréstimo para permitir empréstimos na conta. Neste método vamos controlar o valor das parcelas, para não permitir valores igual ou abaixo de zero;

```
//método emprestimo
public void emprestimo(double valor, int parcelas)
{
    //utilizando o while para validar a qtde de parcelas, deve ser superior a zero (0)
    while(parcelas<=0){
        System.out.print("Valor incorreto para parcelas.");
        parcelas = Teclado.leInt("Informe a qtde de parcelas (1-12): ");
    }

    //criando a variável valorParcela para utilizar no cálculo das parcelas
    double valorParcela = valor/parcelas;

    //utilizando o for para calcular o valor das parcelas com 5% de acrescimo e apresentar para o usuário
    for(int contador = 1; contador <= parcelas; contador++){
        valorParcela += valorParcela*0.05;
        System.out.printf("O valor da parcela %d é: %4.2f \n", contador, valorParcela);
    }

    saldo+=valor;
}
```

6. Método para consultar o valor do saldo da conta;

```
//método consulta saldo
public void consultaSaldo()
{
    System.out.printf("Seu saldo é: %4.2f /n ",saldo);
}
```

7. Método para visualizar as opções que estão disponíveis para a conta corrente.

```
//método opções do caixa eletrônico
public void opcoes()
{
    System.out.println("Bem-vindo ao Caixa Eletrônico!");
    System.out.println("As opções disponíveis são:");
    System.out.println("1 - Saque");
    System.out.println("2 - Depósito");
    System.out.println("3 - Empréstimo");
    System.out.println("4 - Consulta Saldo");
    System.out.println("0 - Encerrar");
}
```

Depois de criar nossa classe ContaCorrente, vamos agora criar a nossa classe de teste, a classe: **CaixaEletronico**. A classe CaixaEletronico vai realizar o seguinte cenário de teste:

1. Criar uma nova conta corrente com o saldo= 600,00;
2. Na sequência, vamos criar uma variável inteira opção, para ler do teclado a opção que o usuário deseja realizar no caixa eletrônico;
3. Como o usuário pode realizar mais de uma operação, vamos utilizar a estrutura de repetição do...while para apresentar, no mínimo uma vez as opções para o usuário. Enquanto o usuário informar opções diferente de zero (0), o sistema permitirá ele realizar qualquer operação disponível;
4. Quando o usuário selecionar uma opção, o sistema deverá validar se a opção selecionada é válida. Para isso vamos utilizar a estrutura de repetição: while;
5. Vamos criar as variáveis auxiliares: valor e parcelas para utilizar na leitura dos valores do teclado;

6. Depois disso, vamos utilizar a estrutura de seleção: switch para comparar qual foi a opção selecionada e chamar o método adequado da ContaCorrente para realizar a operação que o usuário deseja realizar;

7. Por fim, vamos verificar se o usuário deseja realizar uma nova operação testando a condição no final do do...while.

A figura abaixo apresenta a classe de teste: CaixaEletronico com os comandos implementados para cada passo do cenário de teste acima:

```
//classe de teste
public class CaixaEletronico
{
    //método main
    public static void main(String args[])
    {
        //instanciando a ContaCorrente e inicializando o saldo com 600.00
        ContaCorrente conta = new ContaCorrente(600.00);

        //criando a variavel opcao
        int opcao;

        //utilizando do...while para apresentar as opções para o usuário no mínimo uma vez
        do{
            //apresenta as opções do caixa eletrônico
            conta.opcoes();

            //lendo a opção do teclado
            opcao = Teclado.leInt("Informe a opção desejada: ");

            //validando a opção informada com o while
            //enquanto o usuário não informar uma opção correta, o programa solicitará a opção novamente
            while(opcao<0 && opcao>4)
                opcao = Teclado.leInt("Opção inválida! Informe a opção desejada: ");

            //criando as variáveis auxiliares
            double valor;
            int parcelas;
        }
```

```
        //utilizando o switch (estrutura de seleção) para verificar qual foi a opção selecionada
        switch(opcao){
            case 1: //retirada
                valor = Teclado.leDouble("Informe um valor para retirada: ");
                conta.retirada(valor);
                break;
            case 2: // depósito
                valor = Teclado.leDouble("Informe um valor para depósito: ");
                conta.deposito(valor);
                break;
            case 3: // empréstimo
                valor = Teclado.leDouble("Informe um valor para empréstimo: ");
                parcelas = Teclado.leInt("Informe a qtd de parcelas: ");
                conta.emprestimo(valor,parcelas);
                break;
            case 4: //consulta saldo
                conta.consultaSaldo();
            default:
                System.out.println("Encerrando o sistema!");
        }

        }while(opcao!=0); // nas opções o 0 está sendo utilizado para encerrar o programa
    }
}
```

Após compilarmos e executarmos a classe de teste, teremos o seguinte resultado:

```
BlueJ: Terminal Window - PraticaM
Bem-vindo ao Caixa Eletrônico!
As opções disponíveis são:
1 - Saque
2 - Depósito
3 - Empréstimo
4 - Consulta Saldo
0 - Encerrar
Informe a opção desejada: 1
Informe um valor para retirada: 450
Bem-vindo ao Caixa Eletrônico!
As opções disponíveis são:
1 - Saque
2 - Depósito
3 - Empréstimo
4 - Consulta Saldo
0 - Encerrar
Informe a opção desejada: 4
Seu saldo é: 150.00 /n Encerrando o sistema!
Bem-vindo ao Caixa Eletrônico!
As opções disponíveis são:
1 - Saque
2 - Depósito
3 - Empréstimo
4 - Consulta Saldo
0 - Encerrar
Informe a opção desejada: 0
Encerrando o sistema!
```

Enquanto informarmos uma opção diferente de zero (0) será permitido o usuário realizar as operações disponíveis.